

MODULE 03 · HANDS-ON LAB · 60 MIN

模块三 · 核心动手实战 Lab

基于 EnterpriseAgenticWorkshop 源码 · 三阶能力递进实操

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

三个 Lab 顺次递进：从搭流水线到治理再到自愈闭环

01 · LAB 1 · 20 MIN

构建多智能体业务流

拉取模板工程，定义 Planner 与 Worker，跑通结构化消息分发与汇聚。



02 · LAB 2 · 25 MIN

挂载 Harness 中间件

把鉴权、校验、审计、落库、压缩做成可插拔治理中间件。



03 · LAB 3 · 15 MIN

HITL 审批与错误自愈

对敏感工具挂起审批，注入 Token 恢复，异常触发自我修正。

能力叠加：先能跑（多智能体）→ 再能治（Harness 治理）→ 最后能自愈（HITL 与纠错）。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

时间盒锁定 60 分钟，每个 Lab 都有明确产出与验收基线

Lab	时长	核心产出	验收基线
Lab 1 构建多智能体业务流	20 分钟	可运行的 Supervisor + Worker 流水线	消息分发跑通、结果正确 汇聚
Lab 2 挂载 Harness 中间件	25 分钟	拦截 / 落库 / 压缩三件套	审计日志完整、断点可重 放
Lab 3 HITL 审批与错误自愈	15 分钟	审批挂起与自愈闭环	审批恢复、异常自动纠正

验收原则：每个 Lab 必须产出可运行的文件与可复现的现象，而非只看到代码。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

环境地基先打牢：Python 3.12+ 与 Copilot 席位是入场券

PYTHON RUNTIME

Python 3.12+ 作为统一运行时

所有 Lab 代码基于 Python 3.12+；用虚拟环境隔离依赖，避免版本漂移。

COPILLOT SEAT

GitHub Copilot 订阅与 seat

学员账号需已开通 Copilot seat，作为多智能体的内置模型 provider。

```
$ python --version          # 需 >= 3.12
$ python -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
```

先跑通版本校验与依赖安装，再进入任一 Lab；缺少 seat 会直接阻断模型调用。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

两条命令完成 Copilot CLI 安装与登录，取得模型访问权

取得模型访问权

- 安装 gh-copilot 扩展，或使用独立 Copilot CLI
- 执行 auth login，取得内置模型访问权
- 校验版本，确认可调用 Copilot provider

```
# 安装 Copilot CLI 扩展
$ gh extension install github/gh-copilot
# 或使用独立 CLI 登录
$ copilot auth login
# 校验安装
$ gh copilot --version

# 登录后终端显示已认证账号与可用模型
```

登录成功即取得模型访问权，后续各 Lab 直接复用同一 provider，无需重复配置。

配置 Foundry 端点并部署 DeepSeek-V4-Flash 与 gpt-5.5 双模型

PROJECT_ENDPOINT

Foundry 项目端点

在 Azure 订阅创建 Foundry project，取得 endpoint 供 Agent 接入。

MODEL A · DEEPSEEK

DeepSeek-V4-Flash

被测业务模型之一，部署名可用环境变量覆盖。

MODEL B · GPT-5.5

gpt-5.5

第二个被测模型，用于 CLI runner 做快速模型对比。

```
export FOUNDRY_PROJECT_ENDPOINT=https://<your-project>.services.ai.azure.com
export MODEL_A=DeepSeek-V4-Flash      export MODEL_B=gpt-5.5
```

端点与模型名全部走环境变量，切换部署无需改代码，便于在不同模型间对比。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

按模块给齐依赖：Docker、Azure CLI、azd 各司其职

模块	关键依赖	凭据 / 命令
OpenClaw_AgentHarness	Docker + Docker Compose	COPILOT_GITHUB_TOKEN
CodingAgent	Docker + Docker Compose	COPILOT_GITHUB_TOKEN
AgentHarness_HostedAgent	Azure CLI	az login · DefaultAzureCredential
skill-testing-harness-deploy	Azure Developer CLI (azd)	azd up · Foundry 配额

沙箱类模块靠 Docker 与 token；托管与部署类模块靠 Azure 身份与 azd。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

开工前逐项自检，任一红叉都会让 Lab 中途卡死



Python $\geq$ 3.12



Copilot 已登录



Foundry 端点连通



模型部署可见



Docker 守护进程



Azure 凭据有效

六项全绿再开跑；任一红叉都会在 Lab 中途报错而难以定位。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意



LAB 01 · 20 MIN · MULTI-AGENT FLOW

Lab 1 · 构建多智能体业务流

从模板工程到 Supervisor / Worker 结构化协作

Lab1 用 20 分钟产出一条可运行的多智能体业务流

20

分钟时间盒

3

个专职 Agent

4

份工作区产物

1

条自闭环流水线

产出物清单

SPEC.md → solution.py → test_solution.py → RUN_REPORT.md

产出不是“聊天记录”，而是一组可运行、可复现的工作区文件。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

先从 Workshop 拉取模板工程，锁定目录约定再动手

先约定，再动手

Workshop 目录约定清晰：

- /agents — 智能体定义
- /tools — 工具与函数
- /plugins — 语义插件
- /workflows — 编排流程

```
# 拉取模板工程
$ git clone
github.com/kinfey/EnterpriseAgenticWorkshop
$ cd EnterpriseAgenticWorkshop/OpenClaw_AgentHarness
# 启动沙箱栈
$ docker compose up -d

# compose up 后三只 Agent 容器就绪
```

先跑通 clone 与 compose，再定义 Agent；目录约定让工程结构可预测。

把模型端点与基础工具配置为可切换项，避免硬编码

配置项	环境变量	示例值
模型 provider	COPILLOT_MODEL	Claude Opus 4.7 （内置）
Foundry 端点	FOUNDRY_PROJECT_ENDPOINT	https://<proj>.services.ai.azure.com
网关 token	COPILLOT_GITHUB_TOKEN	ghp_xxx （网关注入）
工具白名单	TOOLS_ALLOWED	read, write, edit, exec

端点、凭据、工具全部外置为配置项，换环境不改一行代码。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

划清 Planner/Supervisor 与 Worker/Tool 的职责边界是第一要务

PLANNER / SUPERVISOR

中心协调者

拆解目标为子任务，按能力分发给 Worker，汇聚结果并决定终止。

WORKER / TOOL

专门执行者

只做被分派的单一职责，调用工具后把结构化结果回传上级。

BOUNDARY

边界契约

上下文只读、最小权限、结果经消息契约回传，杜绝越权互调。

先分清谁拆解、谁执行、谁守边界，多智能体才不会互相踩权限。

Supervisor 居中拆解任务，向专门 Worker 分发再汇聚结果

Supervisor · 中心协调者

拆解目标 → 按能力分发 → 汇聚结果 → 决定终止



Worker A · Coder

读 SPEC 写 solution.py



Worker B · Tester

写 test_solution.py



Worker C · Runner

跑 pytest 写 report

↑ 结果经结构化消息契约回传 Supervisor，由其汇聚并判断是否终止。

两种编排各有取舍：Supervisor 中心化与 Peer-to-Peer 交接

SUPERVISOR

中心化编排

中心协调者拆解并分发；可控、可审计，但中心易成瓶颈。

PEER-TO-PEER

对等交接

基于事件 / 消息总线的自适应 Handoff；灵活，但轨迹更难追踪。

选型建议

先中心后对等

Lab 用 Supervisor 起步保证可控；复杂协作再引入 P2P handoff。

先用 Supervisor 建立可控基线，需要弹性交接时再演进到 Peer-to-Peer。

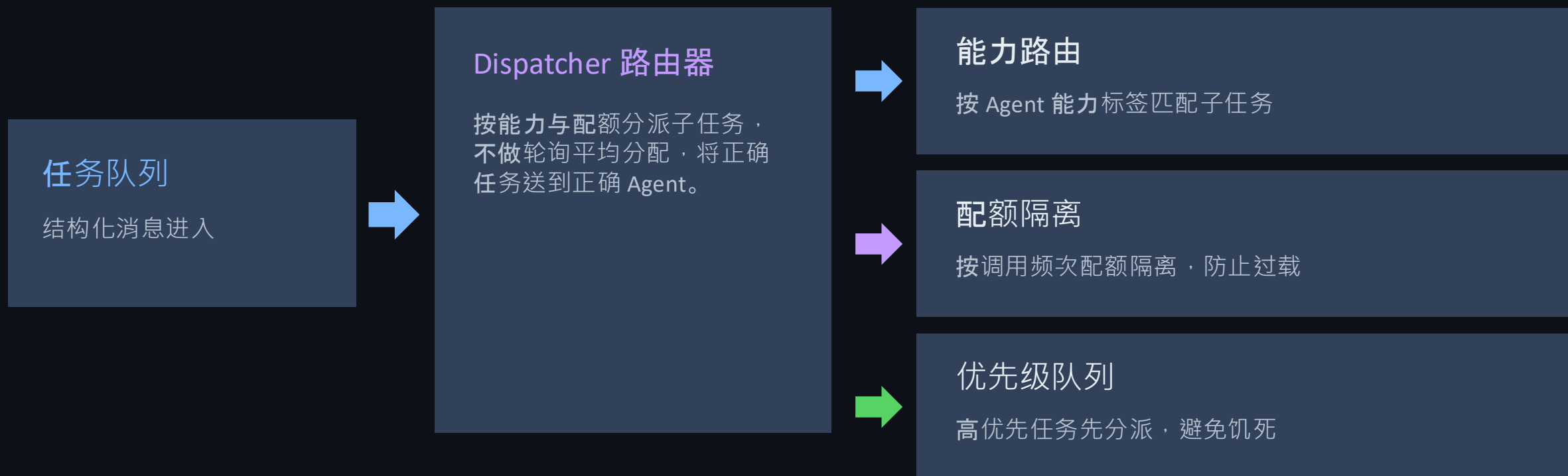
消息契约先定字段：让 Agent 间流转结构化、可校验

字段	类型	说明
task_id	string	任务唯一标识
role	enum	发起 / 接收角色
intent	string	任务意图描述
payload	object	任务数据体
deadline	int	超时时长（秒）
status	enum	pending / done / failed

统一消息契约后，分发、校验、汇聚都能用同一结构无歧义处理。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

分发策略按能力与配额路由，而非轮询平均分配



路由按能力与配额决定去向，而非平均分配，避免热点 Agent 过载。

设定汇聚与终止条件，防止多智能体陷入死循环

AGGREGATOR

结果汇聚器

按 task_id 归并各 Worker 回传，拼装完整结果集。

TERMINATION

终止条件

全部子任务 done 或达到最大轮次即停，避免无限扩散。

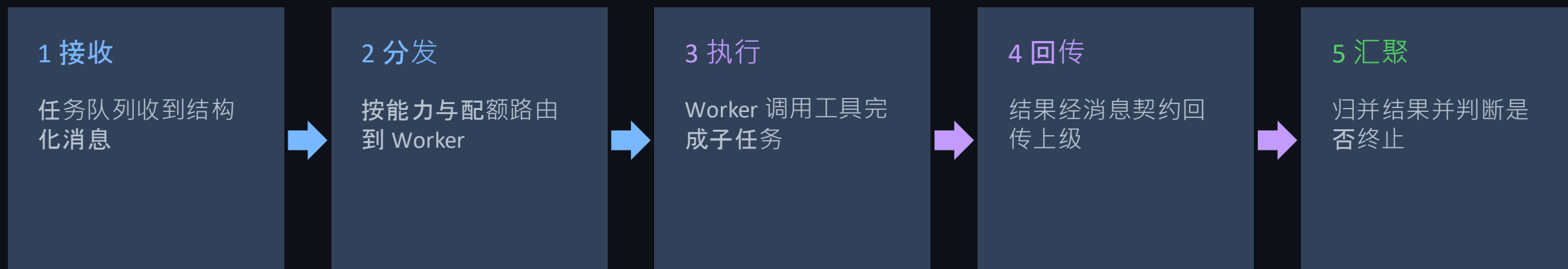
CIRCUIT BREAK

超时熔断

超过 deadline 或反复失败即熔断，交人工或降级处理。

先设清汇聚口径与终止条件，多智能体才不会陷入死循环或空转。

端到端串起来：一条消息从分发到汇聚的完整生命周期



一条消息完整走完接收 → 分发 → 执行 → 回传 → 汇聚，才算跑通业务流。

五个阶段任一断裂，都会让 Lab1 验收不通过。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

用 OpenClaw 三 Agent 落地：Coder、Tester、Runner 各守一职

A · CODER

Coder

读 SPEC.md，写 solution.py；工具白名单 read / write / edit。

B · TESTER

Tester

读 spec + solution，写 test_solution.py；工具 read / write / edit。

C · RUNNER

Runner

沙箱内跑 pytest，写 RUN_REPORT.md；独有 exec 权限。

三个最小权限 Agent 在共享工作区分工协作，只有 Runner 能执行。

共享工作区自闭环：SPEC.md 到 RUN_REPORT.md 一次跑通



🔄 若测试失败，报告回喂下一轮，Coder 依反馈修正——构成自闭环协作。

SPEC.md 到 RUN_REPORT.md 一次跑通，即完成 Lab1 的核心闭环。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

工具白名单把权限收敛到最小，只有 Runner 能执行

CODER

read · write · edit

只能读写工作区文件，不能执行任何命令。

TESTER

read · write · edit

读 spec 与实现并写测试，同样无执行权。

RUNNER

read · write · exec

唯一持 exec，且被 tools.exec.allowedPaths 限制在单一子目录。

把 exec 只授予 Runner 且限定路径，是最小权限落地的关键一步。

Lab1 验收与常见卡点：现象、原因、处置一一对应

现象	原因	处置
Worker 收不到任务	消息契约字段缺失	校验 schema，补齐 role / intent
结果无法汇聚	task_id 不一致	统一 task_id 生成与归并
流程死循环	缺少终止条件	设最大轮次与 deadline 熔断
Coder 执行报错	越权调用 exec	仅 Runner 授 exec，收敛白名单

多数卡点来自契约与权限：先查 schema，再查 task_id，最后查白名单。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意



LAB 02 · 25 MIN · HARNESS MIDDLEWARE

Lab 2 · 挂载 Agent Harness 中间件

把治理能力做成可插拔中间件：拦截·落库·压缩

Lab2 把治理能力做成可插拔中间件，25 分钟挂载三件套

25

分钟时间盒

3

个可插拔中间件

0

行业务逻辑改动

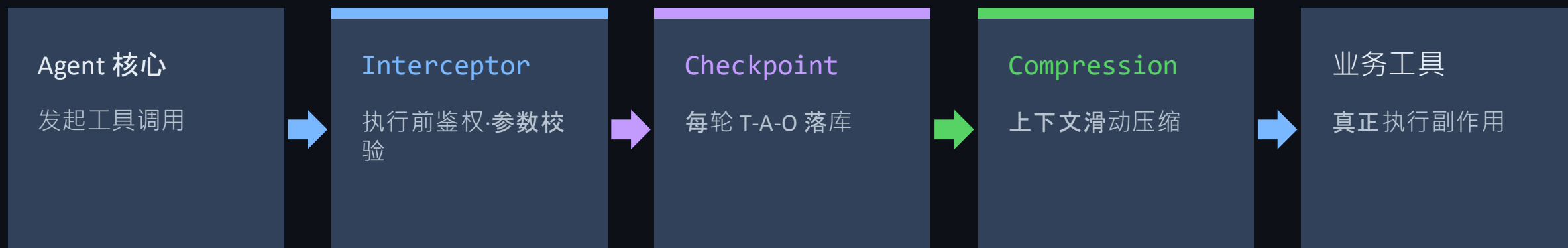
三件治理中间件

`ToolExecutionInterceptor` · `StateCheckpointManager` · `ContextCompressionFilter`

治理不改业务逻辑：三件中间件挂在调用链上，可插拔、可组合。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

治理不改业务逻辑：中间件在调用链上逐层拦截



三件中间件在同一调用链上逐层拦截，任何一层都不触碰业务代码。

拦截、落库、压缩解耦，可分别开关与替换。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

三大中间件分工明确：拦截、落库、压缩各管一段

ToolExecutionInterceptor

工具拦截器

执行前鉴权、参数校验清洗，执行后结构化审计日志。

StateCheckpointManager

状态检查点

每轮 Thought / Action / Observation
实时落库，可断点重放。

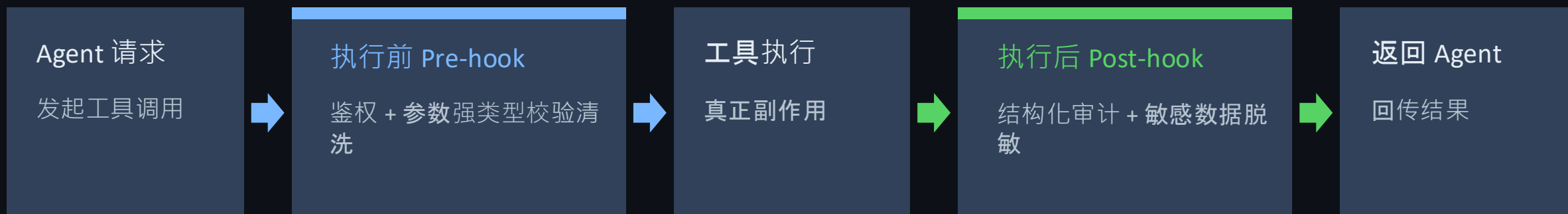
ContextCompressionFilter

上下文压缩

按 token 预算滑动裁剪，保留关键上下文，防止膨胀。

拦截管安全、落库管可恢复、压缩管成本——三段治理互不干扰。

ToolExecutionInterceptor 在执行前后各设一道关卡



同一拦截器包住工具调用两端：执行前挡住越权，执行后留下可审计证据。

两道关卡都不修改工具本身的实现。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

执行前鉴权：先验身份与权限，再决定是否放行工具

先鉴权，后放行

- ▶ 运行时权限绑定：OAuth / Entra ID 身份委托
- ▶ 越权调用在放行前被拒
- ▶ 调用频次配额隔离，防止过载

```
def pre_hook(call, ctx):  
    require_auth(ctx.identity)      # OAuth / Entra ID  
    check_scope(call.tool, ctx.role)  
    if call.rate > quota(ctx):  
        raise QuotaExceeded()  
    return call
```

通过后才返回 call，交给工具执行

把身份、权限、配额三道校验放在工具执行之前，越权请求根本进不去。

参数强类型校验与清洗，挡住 SQL 与 Shell 注入

SCHEMA

强类型校验

按工具 Schema 校验入参类型与取值范围，不合法直接拒绝。

SANITIZE

入参清洗

转义与白名单过滤，防范 SQL / Shell 等注入攻击。

BIND

运行时绑定

参数与身份、配额绑定，越权参数在校验期被拦。

强类型 Schema 加清洗过滤，让注入类攻击在参数层就被挡住。

执行后落审计：Structured Audit Logging 记录可追溯字段

字段	含义	示例
trace_id	全链路追踪 ID	7f3a...c21
agent	发起 Agent	Coder
tool	被调用工具	write_file
args_hash	入参摘要（脱敏）	sha256:9ab...
decision	放行 / 拒绝	allow
ts	时间戳	2026-09-09T00:12Z

审计日志固定字段、结构化落库，事后可按 trace_id 精确回溯每一次调用。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

拦截时序：请求依次经鉴权校验、执行、审计再返回



请求进来后依次经鉴权校验、执行、审计再返回，时序固定可追踪。

任一拍都能在日志里定位到具体工具与决策。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

StateCheckpointManager 把每轮 T/A/O 实时落库可重放

3

段/轮：Thought·Action·Observation

1

份 append-only 日志

0

目标：允许丢失轮次

落库时机

每轮 Thought / Action / Observation 生成即写入检查点存储（append-only）。

每一轮推理都实时落库，崩溃或重启后能从最近检查点无损续跑。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

快照结构固定：一条记录锁住上下文与幂等键

字段	类型	说明
turn_id	int	轮次序号
thought	text	本轮推理
action	json	工具与入参
observation	text	工具返回
idempotency_key	string	幂等键（防重复写）
ts	datetime	落库时间

固定快照结构让每一轮可寻址，幂等键保证重放不产生重复副作用。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

幂等键让重放安全：同一步骤重复执行不再产生副作用

COMPOSE

幂等键组成

由 `turn_id` + `action` 指纹 + 会话 ID 拼成，唯一标识一次操作。

DETECT

冲突检测

写入前对比幂等键，已存在则视为重复，跳过副作用。

REPLAY

安全重放

重启后按幂等键去重写入，重放不会二次执行工具。

幂等键把重复执行变成安全跳过，是断点续跑不出乱子的前提。

重放路径：wake(session_id) 从日志重建状态并续跑

从日志重建状态

- wake(session_id) 读 append-only 日志
- 逐轮回放，幂等键去重
- resume 从最近检查点续跑

```
# 从会话日志恢复
state = wake(session_id)           # 读 append-only 日志
for turn in state.replay():         # 逐轮回放
    if turn.key in applied:         # 幂等去重
        continue
    apply(turn)
resume(state)                       # 从断点续跑

# 去重后从断点续跑，不重复副作用
```

重放不是从头再来，而是按幂等键跳过已完成轮次，从断点精确续跑。

状态流：从运行到检查点，崩溃即从最近快照重放恢复



运行中持续落检查点，一旦崩溃就从最近快照重放并幂等恢复到 RUNNING。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

上下文滑动压缩：给 token 定预算，让长任务不膨胀

把上下文按 token 预算切成三块，动态维持在预算内。

系统提示

固定保留，不裁剪

近期窗口

原样保留最新若干轮

历史摘要

压缩为摘要，按预算裁剪

系统提示固定、近期窗口保留、历史压缩，动态维持在 token 预算内。

预算超限时优先压缩最旧的历史，保住系统提示与近期上下文。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

保留策略分层：系统提示保底、近期原样、历史摘要

SYSTEM

系统提示保底

角色、约束、工具定义常驻，任何时候都不裁剪。

RECENT

近期窗口原样

最新若干轮 Thought / Action / Observation 原样保留。

HISTORY

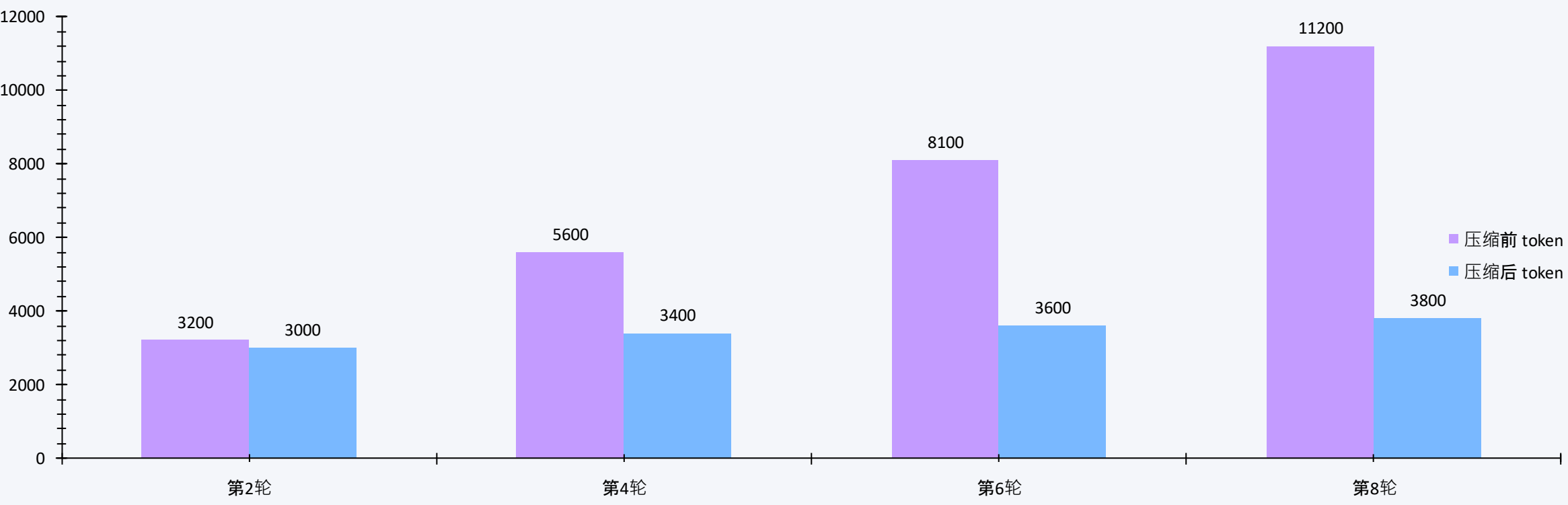
历史分层摘要

更早内容压缩为分层工作记忆摘要，按需再提取。

分层保留策略保证关键信息不丢，同时把历史压进可控的 token 预算。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

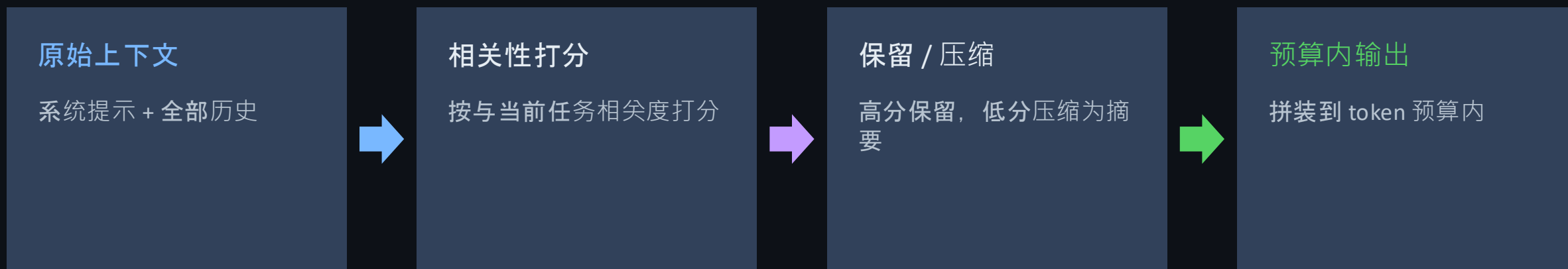
压缩后上下文 token 回落并稳定在预算内 (示意)



压缩前 3.2k → 11.2k 持续膨胀；压缩后稳定在 3.0k~3.8k (token, 示意数据)

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

过滤器管线：原始上下文经打分、取舍后输出到预算内



过滤器像漏斗：先打分再决定保留还是压缩，最终输出恒在 token 预算内。

关键上下文始终留下，冗余历史被折叠成摘要。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

Lab2 验收：三件中间件各自的失败信号都要能读出

中间件	验收信号	常见卡点
ToolExecutionInterceptor	越权被拒、审计完整	未挂钩到调用链，日志缺字段
StateCheckpointManager	断点可重放、幂等	幂等键缺失导致重复副作用
ContextCompressionFilter	token 稳定在预算内	系统提示被误压，关键上下文丢失

验收看失败信号：拦不住、放不回、压过头，任一出现都要立即修。



LAB 03 · 15 MIN · HITL & SELF-HEALING

Lab 3 · HITL 审批与错误自愈

敏感操作挂起审批，异常触发 Agent 自我修正

注册敏感工具 execute_system_deploy, 标记为高危需审批

标记高危工具

- ▶ 用 sensitive=True 标记红线操作
- ▶ 生产部署、资金流转、配置变更属高危
- ▶ Harness 拦截命中即挂起，等待审批

```
@tool(sensitive=True, approval="required")
def execute_system_deploy(target, version):
    """生产部署：高危写入"""
    return deploy(target, version)

# Harness 见 sensitive 即触发审批挂起

# 声明即生效，无需修改业务调用方
```

把生产部署这类红线操作声明为敏感工具，Harness 就能在调用前拦下来。

拦截命中触发挂起：Harness 冻结执行并生成审批请求

MATCH

标记命中

Harness 识别 sensitive 工具调用，判定为红线操作。

SUSPEND

冻结执行

异步挂起当前轨迹，保存状态快照，不真正执行部署。

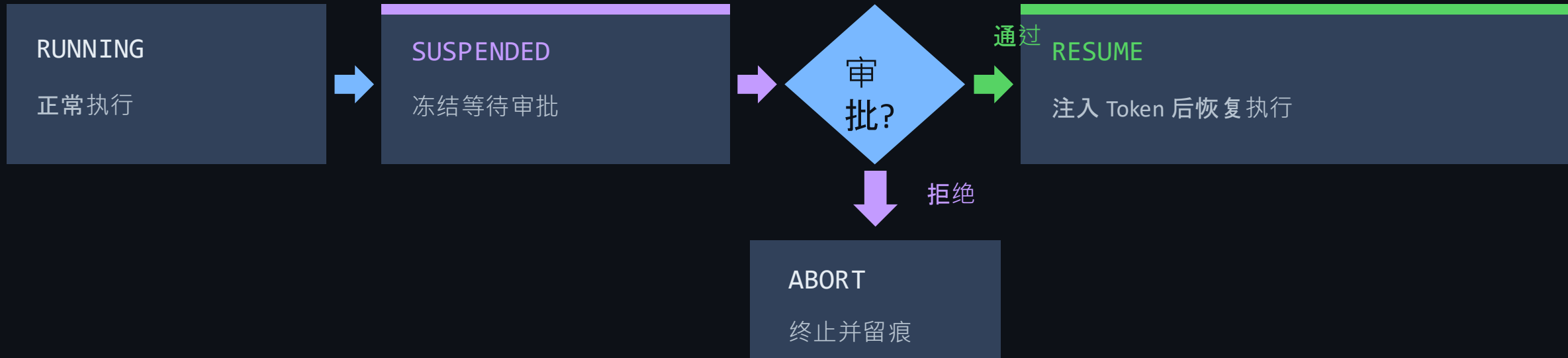
REQUEST

生成审批请求

发出 Approval Webhook，等待人工审批 Token 回注。

拦截命中即冻结并生成审批请求，红线操作在人点头之前绝不落地。

审批状态机：RUNNING 遇敏感即 SUSPEND，等待外部裁决



敏感调用把状态机从运行推入挂起；审批通过才回到执行，否则终止留痕。

注入审批 Token : API/CLI 一条命令解除挂起并恢复执行

外部裁决回注

- ▶ 审批人拿到挂起会话 ID
- ▶ 通过 API 或 CLI 回注审批 Token
- ▶ Harness 校验 Token 后解除挂起

```
# 通过 CLI 审批
$ harness approve --session <id> --token <APPROVAL>

# 或通过 API 回注
$ curl -X POST /approvals/<id> \
      -d '{"decision":"approve"}'

# 校验通过即解除挂起，驱动 Agent 继续
```

审批 Token 从外部注入，Harness 校验通过即解除挂起、驱动 Agent 恢复。

Token 校验通过，Harness 驱动 Agent 从断点恢复执行

VERIFY

Token 校验

核对审批 Token 与会话 ID，防止伪造与重放。

RESTORE

状态恢复

从挂起快照重建轨迹，回到被冻结的那一步。

CONTINUE

继续执行

放行 `execute_system_deploy`，Agent 从断点继续跑完。

校验、恢复、继续三步，让被挂起的敏感操作在获批后无缝续跑。

恢复时序：审批注入后 Harness 唤醒会话并驱动续跑



审批一旦注入，Harness 校验后唤醒会话并从断点续跑，敏感操作才真正执行。

整个 Suspend → Resume 过程对业务代码透明。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

工具抛异常时，Harness 捕获错误并促使 Agent 自我修正

CATCH

异常捕获

工具抛异常时 Harness 拦截，不让错误直接冒泡崩溃。

FEEDBACK

结构化反馈

把 traceback 转成机器可读的错误反馈，回喂给 Agent。

SELF-CORRECT

自我修正

Agent 依反馈调整下一步动作，重试或换方案。

Harness 把异常变成结构化反馈，Agent 据此自我修正，而不是直接崩溃。

对照 CodingAgent：diagnoser 将 traceback 译成 DIAGNOSIS.md 驱动纠错



这就是 CodingAgent 的真实纠错循环：diagnoser 把报错译成 Patch Plan，coder 消费后重跑。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

“

把不确定的大脑，
装进确定的骨架。

— 本模块贯穿的设计哲学：Model 负责思考，Harness 负责治理

MODULE 03 · 沉淀与通向模块四

三个 Lab 沉淀为一套可复用的 Harness 治理心法

- Lab1 多智能体业务流：可运行的 Supervisor / Worker 编排
- Lab2 可插拔 Harness 中间件：拦截、落库、压缩
- Lab3 HITL 审批与错误自愈：可控、可恢复

通向模块四：用 Eval Harness 做自动化评估、可观测性与生产部署。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意